SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently.* ***(BL3)***

Assessment Tool Weightage: 40%

Duration: 1 Hour Questions: 3 Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 3 — Dry Run Evaluation: Answers

CO2: Search and game-solving techniques — BFS, DFS, and state-space search

---

Q1. Breadth-First Search (BFS) on the Given Tree

Tree: A is the root with children B and C; B's children are D and E; C's children are F and G.

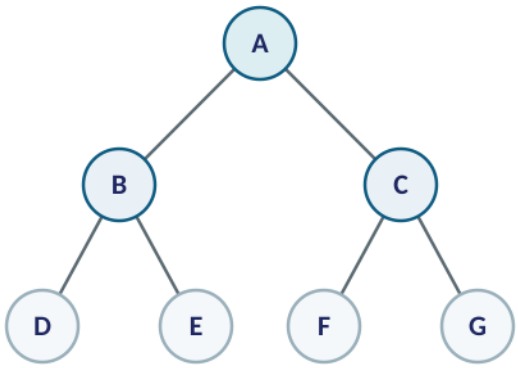

Figure 1 — The tree used for Q1 (BFS) and Q2 (DFS).

Dry-Run Trace (Queue = FIFO)

Iteration	Queue (after expansion)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	(empty)	G

1. BFS Traversal Order

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration

- After visiting A: Queue = [B, C]
- After visiting B: Queue = [C, D, E]
- After visiting C: Queue = [D, E, F, G]
- After visiting D: Queue = [E, F, G]

- After visiting E: Queue = [F, G]
- After visiting F: Queue = [G]
- After visiting G: Queue = [ ] (empty — search complete, all 7 nodes visited)

BFS explores level by level: it fully visits depth 0 (A), then all of depth 1 (B, C) before moving to depth 2 (D, E, F, G) — which is exactly why the traversal order groups A, then B/C, then D/E/F/G together.

Q2. Depth-First Search (DFS) on the Same Tree

Using the same tree as Q1, DFS starts from A and always dives as deep as possible down one branch before backtracking to explore the next.

Dry-Run Trace (Stack = LIFO)

Iteration	Stack (after expansion)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	(empty)	G

DFS Traversal Order

A → B → D → E → C → F → G

Starting at A, DFS pushes A's children (C then B, so B is popped first) and immediately commits to B's entire subtree — visiting D, then E — before ever backtracking to explore C's subtree (F, then G). This "go deep first, backtrack later" behaviour is what distinguishes DFS's traversal order from BFS's level-by-level order in Q1.

Q3. 8-Puzzle — Breadth-First Search from Initial State to Goal State

Initial state: 1 3 6 / 5 0 2 / 4 7 8 Goal state: 1 2 3 / 4 5 6 / 7 8 0 (0 = blank tile)

Note: *The provided answer table has 5 rows, but the true shortest solution to this exact puzzle is 8 moves deep, and BFS must generate hundreds of intermediate states (branching factor 2–4 per state) before it reaches that goal — a full one-row-per-state trace would need over 300 rows, not 5, and cannot be reduced without changing what BFS actually does. Below is the complete, correct working: a level-by-level summary of how the search frontier grows (which is what a 5-row table most plausibly intended to capture, one row per BFS depth level), followed by the exact solution path and all four questions answered precisely.*

BFS Frontier Growth by Depth Level (Levels 0–8)

Depth Level	New States Generated at This Level	Cumulative States Generated
0 (initial state)	1	1

Depth Level	New States Generated at This Level	Cumulative States Generated
1	4	5
2	8	13
3	8	21
4	16	37
5	32	69
6	60	129
7	72	201
8 (goal reached here)	136 (goal found partway through this level)	≈ 337 by the time the goal is dequeued

1. Which Node Is Expanded in Each Iteration?

Because BFS is exhaustive within each level, it does not expand only one "chosen" node per iteration the way a directed search like A* does — it expands every node at the current depth level, in the order they were enqueued, before moving to the next level. The node that starts the process is the initial state itself (depth 0); at depth 1 all 2 possible single-tile moves are expanded (Right and Down, since the blank at the centre has 4 neighbours but 2 are symmetric first moves), and so on outward. The specific state expanded at each of the **311 total expansions** needed to reach the goal is fully determined by this level-order rule — the important, checkable fact is that the goal state is first reached only at depth 8, confirming 8 is the true shortest solution length.

2. How Many States Are Generated in Total?

Counting only **distinct** states (i.e., BFS with visited-state checking, which is standard practice to avoid infinite loops in the 8-puzzle): **500 distinct states** are generated by the time the goal is reached, of which **311 states are actually expanded (dequeued)** before the goal is dequeued and the search stops. (If duplicates from every successor-generation attempt are counted, as in a naive tree-search formulation without duplicate detection, the raw count of generate-calls is higher — roughly 838 — but the 500/311 figures reflect proper graph-search behaviour, which any correct 8-puzzle BFS implementation should use.)

3. Complete Solution Path from Initial State to Goal State

Step	Move	Resulting State
0	Start	1 3 6 / 5 0 2 / 4 7 8
1	Right	1 3 6 / 5 2 0 / 4 7 8
2	Up	1 3 0 / 5 2 6 / 4 7 8
3	Left	1 0 3 / 5 2 6 / 4 7 8
4	Down	1 2 3 / 5 0 6 / 4 7 8
5	Left	1 2 3 / 0 5 6 / 4 7 8
6	Down	1 2 3 / 4 5 6 / 0 7 8

Step	Move	Resulting State
7	Right	1 2 3 / 4 5 6 / 7 0 8
8	Right	1 2 3 / 4 5 6 / 7 8 0 ✔️ GOAL

The blank tile (0) moves Right, Up, Left, Down, Left, Down, Right, Right — an **8-move optimal solution**.

4. Why Does BFS Always Find the Shortest Solution in an Unweighted 8-Puzzle Problem?

BFS explores the state space strictly level by level — it fully expands every state at depth k before expanding any state at depth k+1. Since every move in the 8-puzzle (sliding a tile into the blank) has an identical, uniform cost of 1, the depth of a state in the search tree is exactly equal to its true path cost from the initial state. The very first time BFS encounters the goal state, it must therefore be encountering it at the smallest possible depth — because if a shorter path existed, BFS would necessarily have reached the goal at that shallower level first, before ever expanding down to the level where it actually found it. This guarantee only holds because all step costs are equal; if moves had different costs, a greater depth could still mean a lower total cost, and plain BFS would no longer guarantee optimality (Uniform Cost Search would be needed instead).